Generics

Polymorphism

Poly Morph = Many Shapes

An **object** of **type** `Animal` can be one of several animals, e.g. a `Cat` or a `Giraffe`.

A **method call** `animal.eat()` can have different behavior depending on the runtime type of the animal.

Two Types of Polymorphism

Modern object-oriented languages offer **two types** of polymorphism:

Inheritance (*subtype polymorphism*)

Generics (*parametric polymorphism*)

We need both.

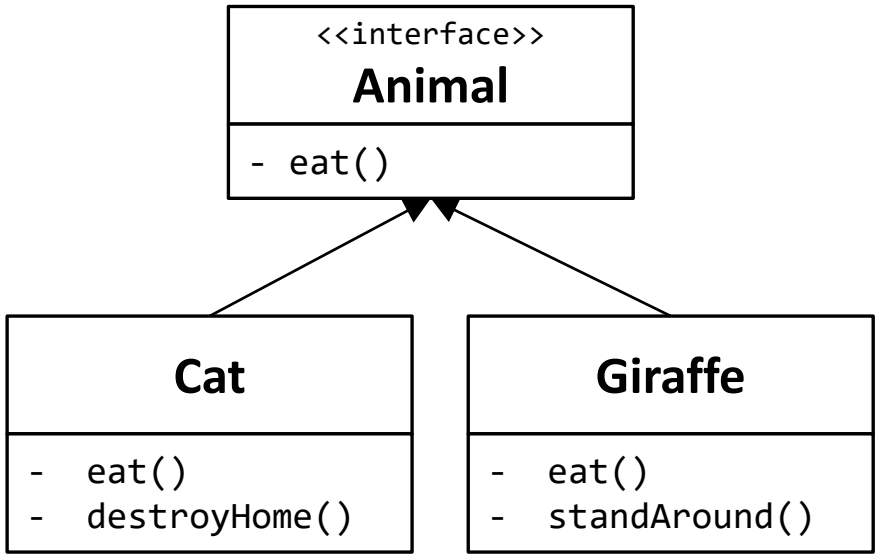
We have already seen subtype polymorphism. Today we shall see generics.

Subtype Polymorphism

Key Idea:

We extract commonalities of several entities into one API.

For example, both a cat and a giraffe eats food. We can create an interface that represents animals and give it an `eat` method. How an animals eat can vary. A cat hunts whereas giraffe forarges.



A Stack of Books

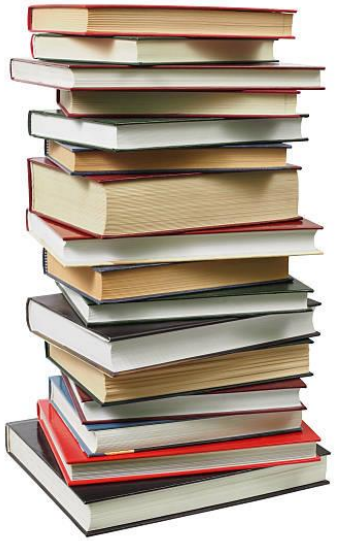
```
public class BookStack {

    private Book[] stack = ...;

    Book pop() {
        // ...
    }

    void push(Book b) {
        // ...
    }

}
```



A Stack of Giraffes

```
public class GiraffeStack {

    private Giraffe[] stack = ...;

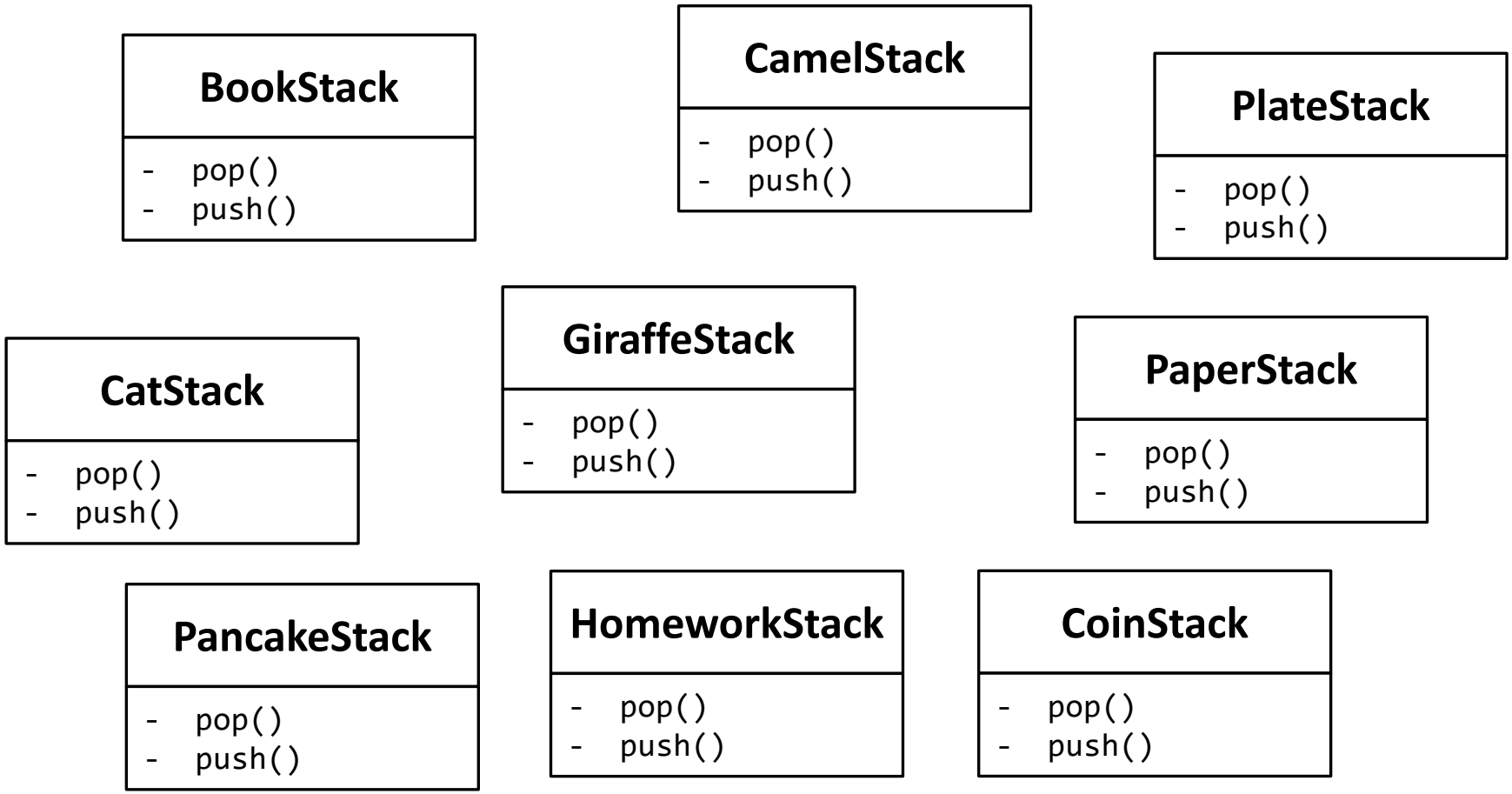
    Giraffe pop() {
        // ...
    }

    void push(Giraffe g) {
        // ...
    }

}
```



A Stack of …


BookStack
- pop()
- push()
CamelStack
- pop()
- push()
PlateStack
- pop()
- push()
CatStack
- pop()
- push()
GiraffeStack
- pop()
- push()
PaperStack
- pop()
- push()
PancakeStack
- pop()
- push()
HomeworkStack
- pop()
- push()
CoinStack
- pop()
- push()

Parametric Polymorphism

Key Idea: Introduce a **type parameter**.

We should not have `BookStack` and `GiraffeStack`, but

`Stack<T>`

where T is a type parameter of `Stack`.

If you want a stack of books you write `Stack<Book>`

If you want a stack of giraffes you write `Stack<Giraffe>`

The types `Stack<Book>` and `Stack<Giraffe>` are different and you cannot accidentally mix them.

Example: Generic Class

```
public class Stack<T> {

    private T[] arr = null;

    T pop() {
        // ...
    }

    void push(T t) {
        // ...
    }

}
```

